

Adopt Library Fundamentals V1 TS Components for C++17 (R1)

There are currently several proposals in flight to adopt portions of the Library Fundamentals Technical Specification for C++17.

The difficulty of such a piecemeal approach is twofold. It adds a lot of unnecessary overhead to process many proposals rather than a single proposal. It may cause useful but unexciting components to get dropped on the floor because no one notices or because no one is willing to go through the drudgery of writing yet another proposal and shepherding it through the committee.

This paper proposes that the components of TS 19568, *C++ Extensions for Library Fundamentals*, listed below be adopted into the C++17 working paper. The list was initialized with all of the components from [N4562, the Library Fundamentals DTS](#), and the LWG/LEWG then struck any components that should not be incorporated directly into C++17. This most recent working paper has identical content to the PDTS, but has a format with HTML anchors that can be directly linked from this paper.

As a follow-up edit, the update to several existing C++17 header synopses is provided directly in this paper, to more clearly specify edits for just the new components being incorporated.

- 3 [General utilities library](#)
 - 3.2 [Tuples](#)
 - 3.2.2 [Calling a function with a tuple of arguments](#)
- 4 [Function objects](#)
 - 4.3 [Searchers](#)
 - 4.3.1 [Class template default_searcher](#)
 - 4.3.1.1 [default_searcher creation functions](#)
 - 4.3.2 [Class template boyer_moore_searcher](#)
 - 4.3.2.1 [boyer_moore_searcher creation functions](#)
 - 4.3.3 [Class template boyer_moore_horspool_searcher](#)
 - 4.3.3.1 [boyer_moore_horspool_searcher creation functions](#)
- 5 [Optional objects](#)
 - 5.1 [In general](#)
 - 5.2 [Header <optional> synopsis](#)
 - 5.3 [optional for object types](#)
 - 5.3.1 [Constructors](#)
 - 5.3.2 [Destructor](#)
 - 5.3.3 [Assignment](#)
 - 5.3.4 [Swap](#)
 - 5.3.5 [Observers](#)
 - 5.4 [In-place construction](#)
 - 5.5 [No-value state indicator](#)
 - 5.6 [Class bad_optional_access](#)
 - 5.7 [Relational operators](#)
 - 5.8 [Comparison with nullopt](#)
 - 5.9 [Comparison with T](#)
 - 5.10 [Specialized algorithms](#)
 - 5.11 [Hash support](#)
- 6 [Class any](#)
 - 6.1 [Header <any> synopsis](#)
 - 6.2 [Class bad_any_cast](#)
 - 6.3 [Class any](#)
 - 6.3.1 [any construct/destruct](#)
 - 6.3.2 [any assignments](#)
 - 6.3.3 [any modifiers](#)
 - 6.3.4 [any observers](#)
 - 6.4 [Non-member functions](#)
- 7 [string_view](#)
 - 7.1 [Header <string_view> synopsis](#)
 - 7.2 [Class template basic_string_view](#)
 - 7.3 [basic_string_view constructors and assignment operators](#)
 - 7.4 [basic_string_view iterator support](#)
 - 7.5 [basic_string_view capacity](#)
 - 7.6 [basic_string_view element access](#)

- 7.7 [basic_string_view modifiers](#)
- 7.8 [basic_string_view string operations](#)
 - 7.8.1 [Searching basic_string_view](#)
- 7.9 [basic_string_view non-member comparison functions](#)
- 7.10 [Inserters and extractors](#)
- 7.11 [Hash support](#)
- 8 [Memory](#)
 - 8.2 [Shared-ownership pointers](#)
 - 8.2.1 [Class template shared_ptr](#)
 - 8.2.1.1 [shared_ptr constructors](#)
 - 8.2.1.2 [shared_ptr observers](#)
 - 8.2.1.3 [shared_ptr casts](#)
 - 8.2.2 [Class template weak_ptr](#)
 - 8.2.2.1 [weak_ptr constructors](#)
 - 8.5 [Class memory resource](#)
 - 8.5.1 [Class memory resource overview](#)
 - 8.5.2 [memory_resource public member functions](#)
 - 8.5.3 [memory_resource protected virtual member functions](#)
 - 8.5.4 [memory_resource equality](#)
 - 8.6 [Class template polymorphic_allocator](#)
 - 8.6.1 [Class template polymorphic_allocator overview](#)
 - 8.6.2 [polymorphic_allocator constructors](#)
 - 8.6.3 [polymorphic_allocator member functions](#)
 - 8.6.4 [polymorphic_allocator equality](#)
 - 8.8 [Access to program-wide memory_resource objects](#)
 - 8.9 [Pool resource classes](#)
 - 8.9.1 [Classes synchronized_pool_resource and unsynchronized_pool_resource](#)
 - 8.9.2 [pool_options data members](#)
 - 8.9.3 [pool_resource constructors and destructors](#)
 - 8.9.4 [pool_resource members](#)
 - 8.10 [Class monotonic buffer resource](#)
 - 8.10.1 [Class monotonic buffer resource overview](#)
 - 8.10.2 [monotonic_buffer_resource constructor and destructor](#)
 - 8.10.3 [monotonic_buffer_resource members](#)
 - 8.11 [Alias templates using polymorphic memory resources](#)
 - 8.11.1 [Header <string> synopsis](#)
 - 8.11.2 [Header <deque> synopsis](#)
 - 8.11.3 [Header <forward_list> synopsis](#)
 - 8.11.4 [Header <list> synopsis](#)
 - 8.11.5 [Header <vector> synopsis](#)
 - 8.11.6 [Header <map> synopsis](#)
 - 8.11.7 [Header <set> synopsis](#)
 - 8.11.8 [Header <unordered_map> synopsis](#)
 - 8.11.9 [Header <unordered_set> synopsis](#)
 - 8.11.10 [Header <regex> synopsis](#)
- 12 [Algorithms library](#)
 - 12.1 [Header <algorithm> synopsis](#)
 - 12.2 [Search](#)
 - 12.3 [Sampling](#)

Directions to the C++ working paper project editor

Add the components in the above list to the C++ *working paper* using the content for each component from [N4562, the Library Fundamentals DTS](#). Move this content from the `std::experimental::fundamentals_v2` inline namespace to the `std` namespace. Strike `experimental/` from the header names, and any additional `#include` directives in those header synopses.

Note: Although we are adopting only the specified Library Fundamentals V1 components into the C++ working paper, the wording from those components comes from Library Fundamentals V2 to ensure the wording includes the latest corrections.

In addition, make the following changed to the header synopsis clauses for the following standard headers, with wording changes relative to [N4567](#), the current C++17 working draft.

17.6.1.2 Headers [headers]

Add the following new headers to Table 14:

- `<any>`
- `<memory_resource>`
- `<optional>`

- `<string_view>`

20.4 Tuples [tuple]

20.4.1 In general [tuple.general]

2 Header <tuple> synopsis

```
namespace std {
    // 20.4.2.9, specialized algorithms:
    template <class... Types>
    void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);
    // 20.4.2.X, Calling a function with a tuple of arguments
    template <class F, class Tuple>
    constexpr decltype(auto) apply(F&& f, Tuple&& t);
}
```

20.9 Function objects [function.objects]

2 Header synopsis

```
namespace std {
    // 20.9.x, Searchers
    template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    class default_searcher;
    template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
             class BinaryPredicate = equal_to<>>
    class boyer_moore_searcher;
    template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
             class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher;

    template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    default_searcher<ForwardIterator, BinaryPredicate>
    make_default_searcher(ForwardIterator pat_first, ForwardIterator pat_last,
                          BinaryPredicate pred = BinaryPredicate());
    template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
             class BinaryPredicate = equal_to<>>
    boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
    make_boyer_moore_searcher(
        RandomAccessIterator pat_first, RandomAccessIterator pat_last,
        Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
    template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
             class BinaryPredicate = equal_to<>>
    boyer_moore_horspool_searcher<RandomAccessIterator, Hash, BinaryPredicate>
    make_boyer_moore_horspool_searcher(
        RandomAccessIterator pat_first, RandomAccessIterator pat_last,
        Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
}
```

Add the following new header synopsis, immediately preceding the additions for [8.5 Class memory resource](#)

20.9.x Polymorphic Memory Resources [memory.polymorphic.resource]

20.9.X.1 Header <memory_resource> synopsis [memory.resource.synop]

```
namespace std::pmr {
    class memory_resource;
    bool operator==(const memory_resource& a,
                    const memory_resource& b) noexcept;
    bool operator!=(const memory_resource& a,
                    const memory_resource& b) noexcept;
    template <class Tp> class polymorphic_allocator;
    template <class T1, class T2>
    bool operator==(const polymorphic_allocator<T1>& a,
                    const polymorphic_allocator<T2>& b) noexcept;
    template <class T1, class T2>
    bool operator!=(const polymorphic_allocator<T1>& a,
                    const polymorphic_allocator<T2>& b) noexcept;
    // Global memory resources
    memory_resource* new_delete_resource() noexcept;
```

```
memory_resource* null_memory_resource() noexcept;
// The default memory resource
memory_resource* set_default_resource(memory_resource* r) noexcept;
memory_resource* get_default_resource() noexcept;
// Standard memory resources
struct pool_options;
class synchronized_pool_resource;
class unsynchronized_pool_resource;
class monotonic_buffer_resource;
} // namespace std::pmr
```
